

Explainable AI for Network Intrusion Detection

Are SHAP explanations of an IDS faithful, stable, and consistent with network protocol semantics?

Mini Research Problem – Advanced Topics in AI & ML

Code and reproduction scripts:

github.com/Wangzx9908/ai-hw-SUMMER-2026-MRP

Motivation

An ML-based IDS outputs *block* or *allow*. A network engineer cannot act on that: was the flow dropped because of a half-open TCP state, a zero-byte payload, or service fan-out? Without a reason there is no way to tell a true detection from a false positive that just killed a production service.

Main idea

Train flow classifiers on NSL-KDD (125 973 train / 22 544 unseen test flows), attribute every decision with SHAP, sum attributions over one-hot groups back to the 41 protocol fields, and then *test the explanations themselves*: domain consistency, faithfulness, stability, cost.

Results

- Learned logic matches textbook signatures: DoS $\rightarrow$ count, flag=REJ; Probe $\rightarrow$ src_bytes=0, diff_srv_rate; R2L $\rightarrow$ hot, logged_in.
- Domain alignment predicts generalisation: lift 1.45 (DoS) $\rightarrow$ 0.83 (U2R), correlation 0.90 with per-family recall on unseen attacks.
- SHAP is markedly more faithful than LIME (AOPC 0.69 vs 0.44, random 0.22) and perfectly reproducible (1.00 vs 0.66 top-5 Jaccard).
- 47 ms per explained flow $\Rightarrow$ viable in a SOC console, not inline at line rate.

The problem: accuracy hides the failure mode

	same-distribution held-out split	official KDDTest+ (17 novel attacks)
Random Forest	99.9 %	74.8 %
MLP (64–32)	99.6 %	79.4 %

Random Forest recall per family on unseen traffic:

DoS	Probe	R2L	U2R
77 %	60 %	7 %	7 %

- A published 99.9 % collapses to 74.8 % once the attack mix changes – the number alone gives no clue where the model went wrong.
- R2L / U2R are essentially undetected, yet the aggregate accuracy still looks acceptable.
- Lecture 09: *interpretability as an ML debugging tool*. We use attributions to ask **which fields the model actually relies on**, and whether those are the ones a protocol analyst would use.

Literature review (1/2): black-box detectors in security

- **Sommer & Paxson, IEEE S&P 2010** – *Outside the Closed World*. ML-based NIDS suffer from a *semantic gap*: an anomaly score is not an actionable finding; operators need the reason to respond. Still the reference framing for this problem.
- **Arp et al., USENIX Security 2022** – *Dos and Don'ts of ML in Computer Security*. Documents spurious correlations and sampling artefacts in security datasets; recommends inspecting learned features rather than trusting held-out scores.
- **Tavallaee et al., CISDA 2009** – NSL-KDD. Removes KDD'99 duplicates and keeps a test split with attack types absent from training, which is exactly the generalisation gap on the previous slide.
- **Rudin, Nature Mach. Intell. 2019** – argues for inherently interpretable models in high-stakes settings instead of post-hoc explanation. Our counter-position: production IDS stacks are already ensembles/DL, so post-hoc attribution must at least be *validated*.
- **Doshi-Velez & Kim, 2017** – taxonomy of interpretability evaluation (functionally / human / application grounded); we stay at the functionally-grounded level (proxy metrics, no user study).

Literature review (2/2): attribution methods and the gap

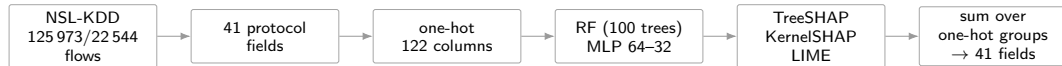
Method	Access	Cost/flow	Determ.	Axioms	Source
LIME	black box	~1k queries	no	none	Ribeiro et al., KDD 2016
KernelSHAP	black box	10^2 – 10^3	no	Shapley (approx.)	Lundberg & Lee, NeurIPS 2017
TreeSHAP	trees	polynomial, exact	yes	Shapley	Lundberg et al., Nat. MI 2020
Integrated Grad.	gradients	~50 passes	yes	completeness	Sundararajan et al., ICML 2017
LEMNA	black box	high	no	none	Guo et al., CCS 2018 (security-specific)

Applied to intrusion detection

- **Wang et al., IEEE Access 2020** – SHAP-based framework for IDS on NSL-KDD / UNSW-NB15: local and global feature importance per attack class.
- **Mane & Rao, 2021** – SHAP + LIME + contrastive explanations on NSL-KDD deep models.
- **Warnecke et al., IEEE EuroS&P 2020** – compares LIME/SHAP/LRP/IG on security tasks using descriptive accuracy, sparsity, stability, efficiency.
- **Han et al., CCS 2021 (DeepAID)** – interprets unsupervised deep anomaly detectors and feeds the explanation back into the detector.
- **Nadeem et al., IEEE EuroS&P 2023** – SoK on XAI for security; notes that most papers *display* explanations without validating them.

Gap addressed here: prior IDS work mostly *shows* SHAP plots. We ask whether the attributions are faithful, reproducible, cheap enough, and aligned with protocol semantics – and whether that alignment says anything about generalisation.

Approach



Key design decision. One-hot encoding turns `service`, `flag`, `protocol_type` into 122 columns, and per-column attributions (`service=ecr_i: 0.004`) are unusable in an incident report. Shapley values are additive, so summing over a disjoint group of columns is exactly the attribution of the original field. All results are reported on the 41 semantic fields.

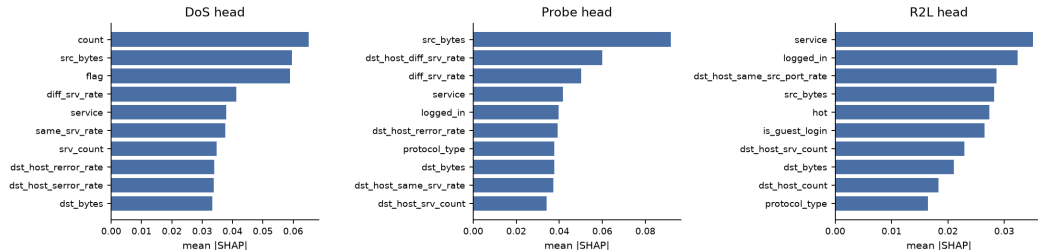
Four experiments on the 2467 explained unseen flows.

- **E1 domain consistency** – share of attributed mass falling in the feature groups (basic / content / time-traffic / host-traffic) that IDS literature associates with each family, *fixed before* looking at any output; reported as lift over a mass-proportional-to-count baseline.
- **E2 faithfulness** – deletion test: move the top- k ranked fields to a benign reference profile, measure the drop in attack probability (120 flows).
- **E3 stability** – re-explain the same flow 5 times, top-5 Jaccard between runs.
- **E4 cost** – wall-clock ms per explained flow.

Tools: [scikit-learn](#) · [shap 0.52](#) · [lime 0.2](#) · data: [NSL-KDD \(UNB CIC\)](#)

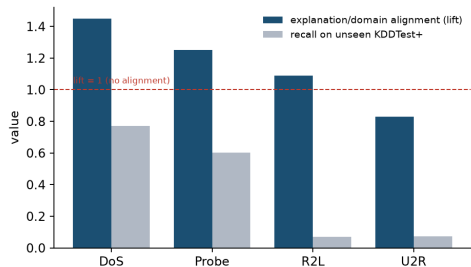
Result 1: global attribution matches protocol intuition

Global attribution on unseen KDDTest+ traffic (41 protocol fields, one-hot groups summed)



- **DoS:** count, flag, srv_count, dst_host_serror_rate – connection volume plus half-open / rejected TCP states, i.e. the SYN-flood signature.
- **Probe:** src_bytes $\approx$ 0 with high diff_srv_rate and dst_host_diff_srv_rate – empty connections fanned out over many services, i.e. a port sweep.
- **R2L:** content/authentication fields logged_in, hot, is_guest_login, service – no volumetric signal at all, consistent with credential attacks hiding inside single well-formed sessions.

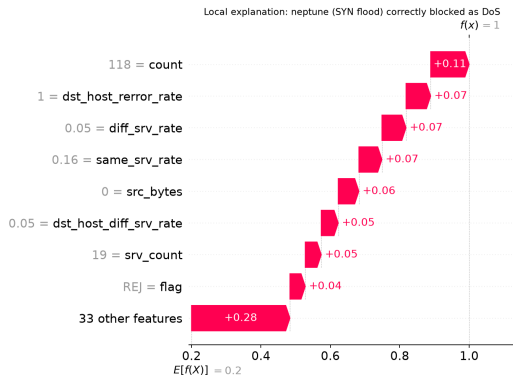
Result 2: alignment with domain knowledge predicts generalisation



Family	expected groups	attributed share	lift
DoS	time+basic	64 % (base 44 %)	1.45
Probe	time+host	58 % (base 46 %)	1.25
R2L	content+basic	58 % (base 54 %)	1.09
U2R	content	26 % (base 32 %)	0.83

- For DoS and Probe the model puts 64 % / 58 % of its attribution mass on the field groups an analyst would use – clear lift over chance.
- For U2R the lift drops *below 1*: the model ignores `root_shell`, `num_file_creations` and leans on host-traffic counters instead – a shortcut, not a signature.
- Alignment lift correlates 0.90 with per-family recall on unseen attacks. The explanation exposed *why* R2L/U2R fail before any new labelled data was collected.

Result 3: a local explanation an engineer can act on



One **neptune** flow, blocked as DoS with $p = 1.00$. Base rate $E[f(x)] = 0.20$.

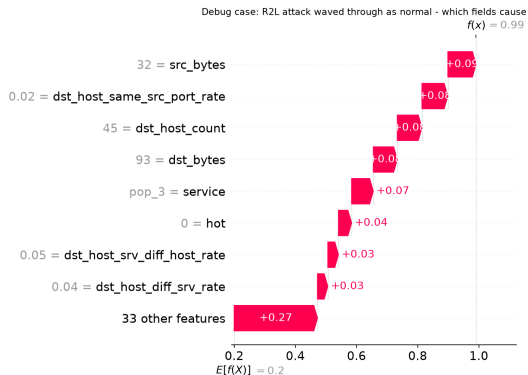
Rendered straight from the SHAP vector as a log line:

```
verdict=DoS p=1.00 because  
count=118 (+0.111),  
dst_host_rerror_rate=1.00 (+0.071),  
diff_srv_rate=0.05 (+0.070),  
same_srv_rate=0.16 (+0.066)
```

118 connections to the same host, all rejected, spread thinly over services: this is the classic SYN flood against closed ports. The model and the protocol analyst agree, and the operator can verify the claim against the flow record in seconds.

Same rendering for a **portsweep**: `src_bytes=0` (+0.110), `dst_host_diff_srv_rate=0.84` (+0.070), `service=private` (+0.067).

Result 4: the same tool as a debugger for a missed attack



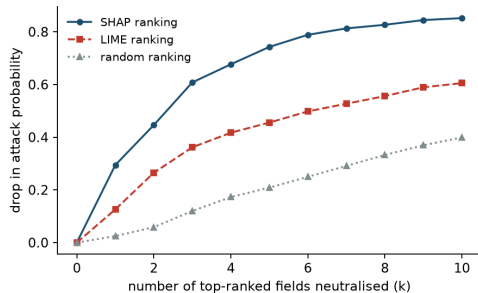
A `guess_passwd` attack waved through as *normal* with $p = 0.99$.

```
verdict=normal p=0.99 because  
src_bytes=32 (+0.092),  
dst_host_same_src_port_rate=0.02 (+0.085),  
dst_host_count=45 (+0.081),  
dst_bytes=93 (+0.078)
```

The verdict rests entirely on *size and host counters* – a small, ordinary looking session. The fields that actually define this attack (`num_failed_logins`, `hot`, `is_guest_login`) contribute almost nothing, because a single credential-guessing connection looks like one legitimate login.

Actionable conclusion, obtained without new labels: per-flow features cannot express R2L. Detection needs **cross-flow aggregation** (failed logins per source over a time window), not a bigger model.

Result 5: which explainer should be trusted? SHAP vs LIME



	TreeSHAP	LIME
Faithfulness (AOPC $\uparrow$)	0.69	0.44
random-ranking baseline	0.22	
Stability, top-5 Jaccard $\uparrow$	1.00	0.66
Cost per flow	47 ms	74 ms

KernelSHAP on the MLP (model-agnostic, 256 samples): 31 ms per flow.

- Neutralising only the **top-3** SHAP fields removes 0.61 of the attack probability; the random ranking needs all 10 to reach less than half of that.
- The two methods largely *disagree* on the same decision (top-5 Jaccard 0.41, Spearman -0.12). Both cannot be right – E2 breaks the tie in favour of SHAP, matching Warnecke et al. (2020).
- LIME's re-sampling makes one in three top-5 fields change between identical runs. An incident report that changes when you press refresh is not evidence.
- Tens of ms/flow: fine for triage in a SOC console or for sampled traffic, far too slow inline on a 10 Gb/s link.

Takeaways and limitations

Takeaways

- Attributions on flow data reproduce textbook attack signatures, which is what makes a block decision defensible to a network engineer.
- Explanations are also a *diagnostic*: alignment lift correlated 0.90 with recall on unseen attacks, and pointed to a concrete fix (cross-flow features for R2L) instead of a bigger model.
- Choose the explainer on evidence, not popularity: exact TreeSHAP was both more faithful (0.69 vs 0.44) and fully reproducible (1.00 vs 0.66).
- Aggregating one-hot columns back to protocol fields costs nothing (additivity) and is what makes the output readable.

Limitations

- NSL-KDD flow features are from 1999 traffic; CICIDS2017 with modern encrypted traffic is the natural next step (loader is the only change).
- The deletion test uses a benign-median reference, which is close to TreeSHAP's implicit background – a mild bias in SHAP's favour; a marginal-resampling variant should be checked.
- Functionally-grounded metrics only; no study of whether operators actually make better decisions with these reasons (Doshi-Velez & Kim).
- Post-hoc explanations can be attacked (Slack et al., AIES 2020) – an adversary aware of the explainer is out of scope here.

Code, data and references

Code (all results in this deck are reproduced by `run_all.sh`)

github.com/Wangzx9908/ai-hw-SUMMER-2026-MRP

- `src/dataset.py` loading, one-hot encoding, group map · `src/train.py` RF + MLP
- `src/explain.py` TreeSHAP global/local figures, reason strings · `src/xai_eval.py` E1-E4 · `src/report.py` figures and slide numbers

Data and libraries

- NSL-KDD: unb.ca/cic/datasets/nsl.html
- CICIDS2017 (future work): unb.ca/cic/datasets/ids-2017.html
- SHAP github.com/shap/shap · LIME github.com/marcotcr/lime · [scikit-learn](https://scikit-learn.org/)
- Interpretable ML book: christophm.github.io/interpretable-ml-book

Key references

- Lundberg & Lee, *A Unified Approach to Interpreting Model Predictions*, NeurIPS 2017.
- Lundberg et al., *From local explanations to global understanding with explainable AI for trees*, Nature Machine Intelligence 2020.
- Ribeiro et al., *"Why Should I Trust You?"*, KDD 2016.
- Sommer & Paxson, *Outside the Closed World*, IEEE S&P 2010.
- Arp et al., *Dos and Don'ts of Machine Learning in Computer Security*, USENIX Security 2022.
- Warnecke et al., *Evaluating Explanation Methods for Deep Learning in Security*, IEEE EuroS&P 2020.
- Wang et al., *An Explainable Machine Learning Framework for Intrusion Detection Systems*, IEEE Access 2020.
- Tavallaee et al., *A Detailed Analysis of the KDD CUP 99 Data Set*, CISDA 2009.
- Nadeem et al., *SoK: Explainable ML for Computer Security Applications*, IEEE EuroS&P 2023.

Thank you

github.com/Wangzx9908/ai-hw-SUMMER-2026-MRP